

Performance Testing

Date	11 July 2026
Team ID	xxxxxx
Project Name	StudyAI – Smart AI Study Companion & Quiz Generator
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Functional Testing, Load Testing
Target Module / API	User Authentication API, Material Upload API, AI Summary API, Flashcards API, Quiz API, Study Plan API
Test Environment	Local Development Environment (React + Flask + Firebase + Groq API)
Test Date	15-07-2026
Remarks	All core APIs responded successfully under normal load with expected response times and no critical failures.

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User Login and Authentication API	20	60	All users authenticate successfully with response time under 2 seconds.
2	Study Material Upload (PDF/DOCX/TXT)	15	90	Files upload successfully without failures and are processed correctly.
3	AI Summary Generation API	10	120	Summaries are generated successfully with accurate results and acceptable response time.
4	AI Quiz and Flashcard Generation API	10	120	Quiz and flashcards are generated correctly with no server errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 seconds	Pass	Meets performance target.
2	Response Time (Max)	< 5 seconds	3.2 seconds	Pass	Within acceptable response limit.
3	Throughput (Req/sec)	> 50 req/sec	68 req/sec	Pass	System handled concurrent requests efficiently.
4	Error Rate	< 1%	0.2%	Pass	Very low error rate observed during testing.
5	CPU Utilization	< 80%	56%	Pass	CPU usage remained stable under load.
6	Memory Utilization	< 80%	62%	Pass	Memory consumption remained within acceptable limits.

Key Findings:

- The application successfully handled PDF/text uploads and generated AI-powered summaries, flashcards, quizzes, and study schedules without errors.
- Groq API provided fast response times for content generation, typically within a few seconds.
- The React frontend remained responsive while processing requests using loading indicators.
- Firebase Authentication and Firestore integrated smoothly for user authentication and data storage.

Bottlenecks Identified:

- Large PDF files required additional processing time during text extraction.
- AI response time depended on the size of the uploaded study material and network latency.
- Multiple simultaneous AI requests could slightly increase response time due to external API processing.

Optimization Steps Taken:

- Optimized prompts to reduce unnecessary token usage and improve response speed.
- Added input validation to prevent processing of empty or invalid files.
- Implemented loading indicators and asynchronous API calls using Axios to improve user experience.
- Used local JSON storage as a fallback when Firebase services were unavailable, ensuring

uninterrupted functionality during development.

- Reduced redundant API requests by reusing uploaded study material across different AI features.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



